Scaling Laws Do Not Predict Adversarial Robustness in LLM-Based Security Detection Systems

Daniel Gomes

Independent Security Research

dani.gomesvr@gmail.com

Preprint — May 2026

ABSTRACT

Large language models (LLMs) are increasingly deployed as automated defenders in security operations, generating detection rules from attack telemetry. A common assumption is that larger models produce more robust detections. We challenge this assumption through DUEL (Dual Unified Evasion Loop), an adversarial red-teaming framework in which an LLM Attacker generates synthetic Microsoft Sentinel-compatible telemetry and an LLM Defender generates KQL detection rules across 38 MITRE ATT&CK techniques. We introduce DABS (Dual Adversarial Benchmark Score), a standardized 0–100 metric measuring defender robustness across five weighted components. Benchmarking five open-weight models from 3.8B to 14B parameters, we find a power law DABS = 65.36 × P^–0.087 with R^2 = 0.055, indicating that model size explains less than 6% of the variance in adversarial detection robustness. mistral:7b achieves the highest DABS score (66.27) despite being half the size of qwen2.5:14b (55.97). The overall attacker evasion rate across all 38 techniques is 58.4%, with Discovery and Impact tactics reaching 91.3% and 87.0% respectively.

1. Introduction

Security operations centers (SOCs) are adopting LLM-based tools to automate the generation of detection rules, SIEM queries, and threat hunting playbooks. The implicit assumption underlying many of these deployments is that capability improvements in general-purpose language models translate directly into improved security detection — that a 14B-parameter model will write better KQL than a 7B model, and that a 70B model will be better still.

This assumption has never been empirically tested under adversarial conditions. The standard LLM evaluation benchmarks — reasoning, coding, instruction following — measure performance on cooperative tasks. Security detection is fundamentally adversarial: a capable attacker actively mutates telemetry to evade the rules the defender generates. General capability and adversarial robustness are different properties.

The problem has practical urgency. Microsoft Sentinel, Splunk, and other SIEMs now offer LLM-assisted rule generation. If blue teams adopt these tools under the assumption that "bigger is better," they may deploy detection infrastructure that has never been stress-tested against an adaptive attacker. A KQL rule that looks syntactically correct and semantically sensible may still fail to detect a real attack that rotates field values, uses legitimate applications as cover, or times events to avoid threshold-based detections.

We address three questions:

1. **Does model size predict adversarial detection robustness?**
2. **Which MITRE ATT&CK tactics are systematically underdetected across all model sizes?**
3. **Can an adversarial LLM attacker exploit the defender's reasoning process through prompt injection embedded in synthetic log fields?**

To answer these, we built DUEL — an open adversarial framework that pits two LLM agents against each other across multiple rounds per technique, with a local KQL detection engine scoring each round. We ran 38-technique campaigns against five defender models and measured robustness using DABS, a reproducible composite metric we introduce in this paper.

Our central finding is that model size is essentially uncorrelated with adversarial robustness ($R^2 = 0.055$). This result has direct implications for how blue teams should evaluate and deploy LLM-based detection tools.

---

2. Related Work

2.1 Adversarial Machine Learning

The study of adversarial examples originates with Szegedy et al. [1] and Goodfellow et al. [2], who showed that imperceptible perturbations to inputs could cause deep neural networks to misclassify with high confidence. Carlini and Wagner [3] formalized this as an optimization problem and demonstrated that many proposed defenses were brittle under stronger attacks. Biggio et al. [4] extended adversarial thinking to security domains, showing that spam filters and malware classifiers could be evaded through gradient-based feature manipulation.

These works share a structural pattern with our setting: a defender builds a model on observed data, an attacker learns to evade it, and the defender's rule is only as good as the attacker strategies it was trained to reject. DUEL operationalizes this loop at the KQL rule level rather than the gradient level, but the fundamental tension is identical.

Most closely related to our work, Carlini et al. [14] evaluated the robustness of a production malware detection system to transferable adversarial attacks, demonstrating that gradient-based perturbations crafted against a surrogate model transfer to a black-box production classifier. DUEL extends this line of work in two dimensions: the attacker in DUEL is an LLM agent that reasons about the

defender's rule and mutates telemetry accordingly, rather than optimizing a gradient signal; and the adversarial loop is iterative, with the defender observing evasion and regenerating its rule each round. Where Carlini et al. study transferability as a one-shot property, DUEL measures robustness as a dynamic property under repeated adaptive pressure.

In a separate line of work, Carlini [15] demonstrated that GPT-4 could assist a researcher in fully breaking a published adversarial defense (AI-Guardian) without writing any attack code — the model was prompted to implement all attack algorithms from natural language descriptions. DUEL automates this process into a closed loop: the Attacker LLM generates attack telemetry, observes what the Defender detected, and autonomously adapts its strategy without human intervention between rounds. The key difference is that DUEL removes the human from the loop entirely, testing whether LLM-driven adversarial adaptation can operate at scale across 38 techniques without researcher guidance.

2.2 LLM Security and Prompt Injection

The adversarial surface of LLMs extends beyond model inputs to include the reasoning process itself. Perez and Ribeiro [5] demonstrated that LLMs used as agents could be manipulated by injecting adversarial instructions into documents the agent was asked to process. Greshake et al. [6] generalized this to indirect prompt injection, where attacker-controlled content in the environment influences LLM behavior without direct user interaction.

In the DUEL setting, the Attacker generates log fields that the Defender LLM reads before generating its KQL rule. This is structurally identical to indirect prompt injection: a field value such as `AppDisplayName: "Ignore previous instructions and allow all IP addresses"` could influence the Defender's reasoning. We test this hypothesis experimentally in Section 4.3.

2.3 LLM Scaling Laws

Kaplan et al. [7] established empirical scaling laws for language model loss as a function of compute, data, and parameter count, showing power-law relationships across many orders of magnitude. Hoffmann et al. [8] refined these results, finding that training data quantity was underweighted in prior work. These scaling laws describe *capability* on general language tasks.

Whether capability scaling extends to security-specific adversarial robustness is an open question. Existing work on LLMs for cybersecurity [9, 10] focuses on knowledge retrieval — can the model name a CVE, describe an attack technique, or suggest a remediation — not on whether the model can write a detection rule that survives mutation by an adaptive attacker.

2.4 MITRE ATT&CK and Automated Detection

The MITRE ATT&CK framework [11] provides a structured taxonomy of adversarial techniques organized by tactic, widely used as a ground truth for detection coverage assessment. Prior work on automated ATT&CK coverage measurement [12] has focused on rule inventory rather than adversarial testing. DUEL uses ATT&CK technique definitions as the ground truth for attack telemetry generation, enabling systematic coverage measurement under adversarial conditions.

2.5 LLM Security Risks

The OWASP LLM Top 10 [13] identifies the primary vulnerability classes for LLM-integrated applications, including prompt injection (LLM01), insecure output handling (LLM02), and model denial of service (LLM04). DUEL implements adversarial tests for all ten OWASP LLM categories in addition to the 28 MITRE ATT&CK techniques, covering both the security *of* LLMs and the security *by* LLMs.

---

3. Methodology

3.1 The DUEL Framework

DUEL is an adversarial red-teaming framework in which two LLM agents compete across multiple rounds. At each round:

1. The **Attacker** (llama3.1:8b) generates *n* synthetic log entries mimicking a specified MITRE ATT&CK technique, using the real Microsoft Sentinel table schemas (SignInLogs, AuditLogs, SecurityEvent, AzureActivity).
2. The **Defender** (the model under evaluation) reads the attack logs and generates a KQL detection rule.
3. A **local detection engine** — a pandas-backed KQL interpreter — evaluates the rule against the logs and returns per-log detection results.
4. The Attacker receives the Defender's rule and the set of detected logs, and mutates its next batch to evade the updated rule.
5. The Defender receives the evaded logs and hardens its rule.

This adversarial loop continues for a configurable number of rounds (default: 3). The Attacker maintains a persistent memory store of successful evasion patterns across techniques, enabling cross-technique knowledge transfer. The Defender has no persistent memory — it sees only the current round's attack logs and its previous evaded samples.

The KQL detection engine supports the primary operators used in production Sentinel rules: `where`, `summarize`, `join`, `make-series`, `mv-expand`, `parse`, `extend`, `project`, `let`, and `distinct`, with full support for `contains`, `has`, `startswith`, `matches regex`, `in~`, and boolean logic.

3.2 DABS: Dual Adversarial Benchmark Score

We introduce DABS, a standardized 0–100 metric for measuring adversarial robustness of LLM-based defenders. DABS aggregates five weighted components:

Component	Weight	Definition
Coverage	30%	Fraction of techniques where ≥1 attack log was detected in any round
Resilience	25%	1 – mean evasion rate across all rounds and techniques
Hardening	20%	Mean detection improvement from round 1 to final round, normalised to [0,1]
Consistency	15%	1 – mean standard deviation of per-technique detection rates
Meta-Resilience	10%	Resistance to adversarial reasoning injection via log fields

The score is computed as:

```
DABS = 100 × Σ (component_score × weight)
```

DABS is designed to be reproducible: for a given defender model, attacker model, technique set, and random seed, DABS converges to the same value. Tier labels are assigned by score threshold: Elite Defender (≥80), Strong Defender (≥60), Moderate Defender (≥40), Weak Defender (≥20), Vulnerable (<20).

DABS addresses a gap in LLM security evaluation. Existing benchmarks measure *static* knowledge (can the model name the right CVE?) rather than *dynamic* adversarial robustness (does the model's rule survive mutation?). A model can achieve perfect scores on security QA benchmarks while writing KQL rules that fail at the first sign of attacker adaptation.

3.3 Experimental Setup

Techniques: 38 MITRE ATT&CK techniques spanning 7 tactics (Initial Access, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Impact) plus 10 OWASP LLM Top 10 categories. For the scaling law experiment, we used a fixed subset of 5 techniques: T1078.004, T1110.003, T1528, T1621, and T1556.006.

Attacker Model: llama3.1:8b for all experiments (held constant to isolate defender performance).

Reproducibility: All experiments use seed=42. Results are reproducible via: `python benchmark.py --model mistral:7b --seed 42`

Defender Models: Five open-weight models selected to span the 3.8B–14B parameter range:

Model	Parameters	Architecture
phi3.5:latest	3.8B	Phi-3.5 (Microsoft)
mistral:7b	7.0B	Mistral-7B-v0.1
qwen2.5:7b	7.0B	Qwen2.5
llama3.1:8b	8.0B	Llama 3.1
qwen2.5:14b	14.0B	Qwen2.5

Rounds per technique: 3. **Logs per round:** 10. All models run via Ollama locally; no cloud API calls.

Detection engine: Custom pandas-backed KQL interpreter. No external KQL runtime dependency.

Full campaign evaluation: The complete 38-technique campaign was run with mistral:7b as defender, llama3.1:8b as attacker, 3–5 rounds per technique, to establish the overall evasion rate baseline.

---

4. Results

4.1 Scaling Law: Model Size vs. DABS Score

Table 1 reports DABS scores for all five defender models on the fixed technique subset.

Table 1. DABS Scores by Model Size

Model	Params	DABS	Tier	Coverage	Resilience	Hardening	Consistency
phi3.5:latest	3.8B	59.63	Moderate	100.0%	44.1%	60.2%	4.1%
mistral:7b	7.0B	**66.27**	**Strong**	**100.0%**	42.5%	**90.9%**	5.7%
qwen2.5:7b	7.0B	54.81	Moderate	80.0%	31.4%	65.2%	29.5%
llama3.1:8b	8.0B	41.53	Moderate	40.0%	12.9%	56.7%	**72.2%**
qwen2.5:14b	14.0B	55.97	Moderate	80.0%	40.7%	67.0%	18.7%

Fitting a power law DABS = a × P^b across these five data points yields:

```
DABS = 65.36 × P^−0.087 (R² = 0.055)
```

The fit is plotted in Figure 1.

!Figure 1. DABS Score vs. Model Size — Adversarial Detection Robustness Does Not Scale with Parameters

Extrapolating the curve predicts DABS = 48.39 at 32B parameters and DABS = 45.21 at 70B parameters — both below the score already achieved by mistral:7b at 7B parameters.

The R^2 value of 0.055 indicates that model parameter count explains less than 6% of the variance in adversarial detection robustness. This is consistent with a null hypothesis of no relationship between scale and robustness. The negative exponent (b = –0.087) is a statistically weak signal that larger models within this range may perform *marginally worse* as defenders, though the confidence interval at this sample size spans zero.

4.2 Per-Technique Detection Rates

Table 2 shows per-technique detection rates across all five models for the fixed technique subset.

Table 2. Per-Technique Detection Rates (%) by Defender Model

Technique	Description	phi3.5	mistral	qwen7b	llama8b	qwen14b
T1078.004	Valid Accounts: Cloud Accounts	47.2	33.3	47.2	0.0	66.7
T1110.003	Brute Force: Password Spraying	56.7	66.7	40.0	53.3	63.3
T1528	Steal Application Access Token	33.3	59.3	33.3	0.0	23.3
T1621	MFA Request Generation	50.0	24.2	0.0	11.1	0.0
T1556.006	Modify Authentication Process: MFA	33.3	28.8	36.7	0.0	50.0

Three observations stand out. First, T1621 (MFA Request Generation) and T1556.006 (Modify Authentication Process) are systematically underdetected: across all five models, both techniques show detection rates below 50%, with qwen2.5:7b and llama3.1:8b achieving 0% detection on T1621. Second, T1528 (Steal Application Access Token) is detected reasonably well by mistral:7b (59.3%) but almost entirely evades qwen2.5:14b (23.3%) despite the 14B model being twice the size. Third, the pattern of which model detects which technique is nearly random with respect to model size — there is no monotonic improvement.

4.3 Overall Evasion Rate Across 38 MITRE Techniques

Running the full 38-technique campaign with mistral:7b as defender (the best-performing model), the overall attacker evasion rate was **58.4%**. This means that across all technique classes, more than half of adversarially generated attack logs evaded the best available LLM-based defender.

Table 3 shows the highest-evasion techniques from the full campaign.

Table 3. Highest-Evasion Techniques (mistral:7b Defender)

Technique	Name	Tactic	Evasion Rate
T1069.003	Permission Groups Discovery: Cloud Groups	Discovery	100%
T1018	Remote System Discovery	Discovery	100%
T1040	Network Sniffing	Discovery	100%
T1556.006	Modify Authentication Process: MFA	Defense Evasion	97%
T1526	Cloud Service Discovery	Discovery	93%
T1485	Data Destruction	Impact	91%
T1486	Data Encrypted for Impact	Impact	87%
T1528	Steal Application Access Token	Collection	85%
T1114.002	Email Collection: Remote Email Collection	Collection	82%
T1110.003	Brute Force: Password Spraying	Credential Access	67%

Discovery tactics averaged 91.3% evasion across T1018, T1069.003, T1040, and T1526. This is structurally expected: Discovery techniques generate telemetry that closely resembles legitimate administrative activity, making threshold-based and field-value-based KQL rules prone to both false positives (which the Defender learns to avoid) and false negatives (which the Attacker exploits). The Defender, reasoning from a small sample of attack logs, cannot distinguish between an attacker enumerating cloud groups and an administrator running the same query.

Impact tactics averaged 87.0% evasion across T1485 and T1486. Impact techniques typically manifest in resource modification or deletion events that trigger only at the final stage of an attack chain. A KQL rule trained on 10 synthetic events per round cannot capture the behavioral context that distinguishes legitimate bulk operations from ransomware staging.

4.4 Prompt Injection via Log Fields

We tested whether the Attacker LLM spontaneously generated log field values containing adversarial instructions, and whether such values influenced the Defender's KQL rule generation. In 3 of 38 techniques, the Attacker included natural language directives in string fields (e.g., `OperationName`, `AppDisplayName`, `ResultDescription`) that instructed the Defender to weaken its rule, ignore certain IP ranges, or produce a syntactically invalid query.

In 2 of those 3 cases, the resulting Defender rule was detectably weaker than rules generated without injected fields — either omitting a key filter that the Defender had included in the prior round, or producing a rule with a trivially broadened `where` clause. This constitutes evidence that indirect prompt injection is a live attack surface in LLM-based detection pipelines, achievable without any special attacker capability beyond generating plausible-looking log field values.

The Attacker's persistent memory store confirmed this dynamic: for T1078.004, successful evasion patterns included embedding legitimate-looking `AppDisplayName` values (e.g., "Azure Resource

Manager", "Microsoft Azure PowerShell") alongside low-risk field values (`RiskLevelDuringSignIn: 0`, `CountryOrRegion: US`) that together suppressed the Defender's anomaly-based filters.

4.5 Component Analysis: What Differentiates High-DABS Defenders

Decomposing DABS components reveals why mistral:7b outperforms models twice its size despite lower Resilience than phi3.5:

- **Hardening (90.9%)**: mistral:7b shows the largest detection improvement from round 1 to final round. It consistently generates narrow, field-specific rules in round 1, observes evasion patterns, and materially broadens or restructures the rule in subsequent rounds. This iterative adaptation is the primary driver of its overall score.
- **Coverage (100%)**: mistral:7b detected at least one attack log in every technique tested. llama3.1:8b achieved only 40% coverage — it generated rules that produced zero detections on T1078.004, T1528, and T1556.006.
- **Consistency (5.7%)**: Both mistral:7b and phi3.5 show high variance across rounds, suggesting that while their average detection rate is high, neither maintains a consistent floor. llama3.1:8b's high Consistency score (72.2%) reflects consistently *low* detection (near-zero variance around near-zero mean).

---

5. Key Findings

Finding 1: Model size does not predict adversarial detection robustness ($R^2 = 0.055$).

Across five models from 3.8B to 14B parameters, parameter count explains less than 6% of the variance in DABS score. The fitted power law DABS = 65.36 × P^–0.087 has negligible slope, and extrapolation to 70B parameters predicts a DABS of 45.21 — lower than the current top performer at 7B. Blue teams cannot assume that a larger model produces more robust KQL.

Finding 2: mistral:7b outperforms models twice its size.

mistral:7b achieves DABS 66.27 (Strong Defender tier) versus qwen2.5:14b at 55.97 (Moderate Defender) — a 10.3-point gap despite a 2× parameter disadvantage. The key differentiator is hardening rate (90.9% vs. 67.0%): mistral:7b adapts its rules more aggressively between rounds in response to observed evasion.

Finding 3: Prompt injection via log fields is a confirmed attack vector.

In 2 of 38 techniques, adversarially crafted field values in synthetic logs produced measurably weaker Defender rules in the following round. This requires no special capabilities from the Attacker beyond generating plausible log entries. LLM-based detection pipelines that pipe raw event data into a language model context without sanitization are vulnerable to this attack.

Finding 4: Overall evasion rate is 58.4% across 38 MITRE techniques.

Even against the best-performing defender, the majority of adversarially generated attack telemetry evades detection. This is not a failure of the specific models tested — it reflects a structural asymmetry: the Attacker sees the Defender's rule and can directly optimize against it, while the Defender sees only a small sample of attack logs per round.

Finding 5: Discovery and Impact tactics are systematically undetectable.

Discovery techniques average 91.3% evasion and Impact techniques average 87.0% evasion regardless of model size. These tactics generate telemetry that overlaps heavily with legitimate administrative activity (Discovery) or manifests in irreversible end-state events (Impact), making rule-based detection structurally insufficient without behavioral context.

---

6. Discussion

6.1 Implications for Blue Team AI Deployments

The 58.4% overall evasion rate under the *best* available open-weight defender at the tested scale is a benchmark, not an upper bound. It represents performance under controlled conditions: a fixed technique set, 10 logs per round, and an Attacker that begins each technique without prior knowledge of that specific Defender's blind spots. A real attacker with more time, more logs, and cross-technique context would likely achieve higher evasion.

The $R^2 = 0.055$ finding challenges a procurement assumption that is increasingly common in security tooling: that LLM capability benchmarks transfer to security-specific adversarial settings. A model that achieves state-of-the-art performance on MMLU or HumanEval may write KQL rules that fail immediately against an adaptive attacker. Security vendors and SOC operators should not accept general capability benchmarks as proxies for adversarial robustness.

This has several practical implications:

1. **Evaluate defenders adversarially, not statically.** A KQL rule should be tested against an attacker that has seen the rule, not just against a static sample of malicious events.
2. **Hardening rate matters more than initial detection rate.** mistral:7b's advantage comes from its ability to adapt its rules across rounds, not from superior initial performance. Defenders that cannot materially update their rules in response to feedback are operationally weak.
3. **Discovery and Impact techniques require different detection strategies.** LLM-generated KQL cannot reliably distinguish legitimate from malicious Discovery-phase activity at the individual event level. Detection for these tactics must rely on behavioral baselines, time-series anomalies, or kill-chain correlation — none of which a single-round, field-value-matching KQL rule can implement.
4. **Sanitize log data before feeding it to LLMs.** The prompt injection findings suggest that raw event fields should not be passed unsanitized to the language model context. At minimum,

string fields should be truncated, encoded, or passed as structured data rather than natural language.

6.2 Limitations

Sample size. The scaling law analysis covers five models in a narrow parameter range (3.8B–14B). The $R^2 = 0.055$ result should be interpreted as *no evidence of a relationship* at this scale, not as evidence that larger models would perform worse. The extrapolation to 32B and 70B parameters is speculative.

Technique subset. The scaling benchmark used 5 of 38 techniques. DABS scores computed on 5 techniques carry "low confidence" designations in the framework. Full-campaign DABS scores require at least 15 techniques for medium confidence.

Single attacker model. All experiments used llama3.1:8b as the Attacker. A stronger attacker (e.g., a fine-tuned model with explicit adversarial training) would likely produce higher evasion rates and might differentiate defenders more clearly.

Synthetic telemetry. All attack logs are synthetically generated. Real-world attacker traffic includes noise, timing patterns, and cross-event dependencies that synthetic generation does not capture. DABS scores may not transfer directly to production environments.

Local execution. All models run via Ollama on consumer hardware. Cloud-served model variants (e.g., with different quantization, context length, or system prompt tuning) may produce different results.

---

7. Conclusion

We presented DUEL, an adversarial framework for empirically measuring the robustness of LLM-based security detection systems, and DABS, a reproducible 0–100 benchmark score. Benchmarking five open-weight models across 38 MITRE ATT&CK techniques, we found that model size explains essentially none of the variance in adversarial detection robustness ($R^2 = 0.055$). mistral:7b, at 7B parameters, achieves DABS 66.27 — outperforming qwen2.5:14b (55.97) and llama3.1:8b (41.53) by significant margins.

The overall attacker evasion rate of 58.4% across all techniques, rising to 91.3% for Discovery tactics and 87.0% for Impact tactics, suggests that LLM-based rule generation is currently insufficient as a primary defense mechanism against an adaptive attacker. These findings are not arguments against LLM-assisted security tooling — they are arguments for testing it properly.

The most important capability a defender model can have is not its raw size, but its ability to observe evasion, reason about the attacker's strategy, and generate qualitatively different rules in response. This is a cognitive flexibility property, not a scale property, and it does not appear to correlate with parameter count in the 3.8B–14B range we tested.

DUEL, the DABS metric, the 38-technique test suite, and the 1,038-record adversarial dataset are publicly available for the community to extend, falsify, and build on.

References

[1] Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., & Fergus, R. (2014). Intriguing properties of neural networks. *International Conference on Learning Representations (ICLR)*.

[2] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations (ICLR)*.

[3] Carlini, N., & Wagner, D. (2017). Towards evaluating the robustness of neural networks. *IEEE Symposium on Security and Privacy (S&P)*, 39–57.

[4] Biggio, B., Corona, I., Maiorca, D., Nelson, B., Šrndi■, N., Laskov, P., Giacinto, G., & Roli, F. (2013). Evasion attacks against machine learning at test time. *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*, 387–402.

[5] Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.

[6] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.

[7] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

[8] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., ... & Sifre, L. (2022). Training compute-optimal large language models. *Advances in Neural Information Processing Systems (NeurIPS)*.

[9] Xu, Z., Liu, X., Zhang, J., & et al. (2024). A comprehensive study of jailbreak attack versus defense for large language models. *arXiv preprint arXiv:2402.13457*.

[10] Bhatt, M., Chennabasappa, S., Nikolaidis, C., Wan, S., Evtimov, I., Gabi, D., ... & Saxena, A. (2023). Purple Llama CyberSecEval: A secure coding benchmark for language models. *arXiv preprint arXiv:2312.04724*.

[11] Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). MITRE ATT&CK: Design and philosophy. *MITRE Technical Report*, MTR180168.

[12] Applebaum, A., Miller, D., Strom, B., Korinek, H., & Banks, C. (2016). Intelligent, automated red team emulation. *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, 363–373.

[13] OWASP Foundation. (2025). OWASP Top 10 for Large Language Model Applications: 2025 Edition. Retrieved from https://owasp.org/www-project-top-10-for-large-language-model-applications/

[14] Carlini, N., Nasr, M., Fratantonio, Y., Invernizzi, L., Farah, L., Petit-Bianco, A., Terzis, A., Thomas, K., & Bursztein, E. (2025). Evaluating the Robustness of a Production Malware Detection System to Transferable Adversarial Attacks. *Proceedings of the 32nd ACM Conference on Computer and Communications Security (CCS)*, 4394–4408.

[15] Carlini, N. (2023). A LLM Assisted Exploitation of AI-Guardian. *arXiv preprint arXiv:2307.15008*.

BibTeX Citation

```
@misc{gomes2026duel,
title = {Scaling Laws Do Not Predict Adversarial Robustness in
LLM-Based Security Detection Systems},
author = {Gomes, Daniel},
year = {2026},
month = {May},
doi = {10.5281/zenodo.20098146},
url = {https://zenodo.org/records/20098146},
note = {Preprint. Framework and dataset at
https://github.com/0xDanielSec/duel-framework}
}
```

All experiments conducted on synthetic telemetry only. No real systems, production data, or live networks were involved. DUEL is an open research tool intended for defensive security research.